

```
-----  
-----  
-- Engineer: Olasubomi Borishade  
--  
-- Create Date: 06/01/2026 11:00:22 PM  
-- Design Name: Systolic Array  
-- Module Name: Systolic_Array - Behavioral  
-- Project Name: Parameterized Matrix Multiply Engine  
  
-- Description: A grid of interconnected processing units  
-- that pass data to one another in a pipelined fashion.  
  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
-----
```

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.NUMERIC_STD.ALL;  
use WORK.ARRAY_CUSTOM_PACK.ALL;
```

```
-- Uncomment the following library declaration if instantiating  
-- any Xilinx leaf cells in this code.  
--library UNISIM;  
--use UNISIM.VComponents.all;
```

```
entity Systolic_Array is
```

```
    Generic (    Rows          : INTEGER := 2;  
                Cols          : INTEGER := 2;  
                Input_Width   : INTEGER := 8;  
                Accumulator_Width : INTEGER := 32  
                );
```

```

Port      (      Input_A          : in   SignedArray_1d (0 to Rows
- 1) (Input_Width - 1 downto 0);
          Input_B                  : in   SignedArray_1d (0 to Cols
- 1) (Input_Width - 1 downto 0);
          Clock                    : in   STD_LOGIC;
          Reset                    : in   STD_LOGIC;
          Enable                   : in   STD_LOGIC;
          Clear_Accumulator        : in   STD_LOGIC;
          Output                   : out  SignedArray_2d (0 to Rows
- 1) (0 to Cols - 1) (Accumulator_Width - 1 downto 0)
          );

```

```

end Systolic_Array;

```

architecture Behavioral of Systolic_Array is

```

Signal A_Skew      : SignedArray_2d(0 to Rows - 1) (0 to Rows -
1) (Input_Width - 1 downto 0);
Signal B_Skew      : SignedArray_2d(0 to Cols - 1) (0 to Cols -
1) (Input_Width - 1 downto 0);
Signal A_Horizontal : SignedArray_2d(0 to Rows - 1) (0 to Cols -
1) (Input_Width - 1 downto 0);
Signal B_Vertical   : SignedArray_2d(0 to Rows - 1) (0 to Cols -
1) (Input_Width - 1 downto 0);

```

```

begin

```

```

Gen_A_Row : for i in 0 to Rows - 1 generate
  Gen_A_Stage : for stage in 0 to Rows - 1 generate

```

```

    Gen_A_Stage_0 : if stage = 0 generate
      A_Skew(i)(0) <= Input_A(i);
    end generate;

```

```

    Gen_A_Stage_N : if stage > 0 generate
      process(Clock)
      begin

```

```

        if rising_edge(Clock) then
            if Reset = '1' then
                A_Skew(i)(stage) <= (others => '0');
            else
                A_Skew(i)(stage) <= A_Skew(i)(stage - 1);
            end if;
        end if;
    end process;
end generate;

end generate;
end generate;

Gen_B_col : for j in 0 to Cols - 1 generate
    Gen_B_stage : for stage in 0 to Cols - 1 generate

        Gen_B_Stage_0 : if stage = 0 generate
            B_Skew(j)(0) <= Input_B(j);
        end generate;

        Gen_B_Stage_N : if stage > 0 generate
            process(Clock)
            begin
                if rising_edge(Clock) then
                    if Reset = '1' then
                        B_Skew(j)(stage) <= (others => '0');
                    else
                        B_Skew(j)(stage) <= B_Skew(j)(stage - 1);
                    end if;
                end if;
            end process;
        end generate;
    end generate;
end generate;

```

```
end generate;
```

```
Gen_PE_row : for row in 0 to Rows - 1 generate
```

```
    Gen_PE_col : for col in 0 to Cols - 1 generate
```

```
        Gen_Left_Edge : if col = 0 generate
```

```
            Gen_Top_Left : if row = 0 generate
```

```
                PE_Inst : entity
```

```
work.Processing_Element(Behavioral)
```

```
                Generic map ( Input_Width      => Input_Width,  
                               Accumulator_Width =>
```

```
Accumulator_Width )
```

```
                Port map      ( Clock          => Clock,  
                               Reset           => Reset,  
                               Enable          => Enable,  
                               Clear_Accumulator =>
```

```
Clear_Accumulator,
```

```
                               Input_A         =>
```

```
A_Skew(row) (row) ,
```

```
                               Input_B         =>
```

```
B_Skew(col) (col) ,
```

```
                               Output_A        =>
```

```
A_Horizontal(row) (col) ,
```

```
                               Output_B        =>
```

```
B_Vertical(row) (col) ,
```

```
                               Output          =>
```

```
Output(row) (col)
```

```
                );
```

```
            end generate;
```

```
        Gen_Left_Interior : if row > 0 generate
```

```
            PE_Inst : entity
```

```
work.Processing_Element(Behavioral)
```

```

        Generic map ( Input_Width      => Input_Width,
                      Accumulator_Width =>
Accumulator_Width )

        Port map      ( Clock           => Clock,
                      Reset             => Reset,
                      Enable            => Enable,
                      Clear_Accumulator =>
Clear_Accumulator,

                      Input_A          =>
A_Skew(row)(row),

                      Input_B          =>
B_Vertical(row - 1)(col),

                      Output_A         =>
A_Horizontal(row)(col),

                      Output_B         =>
B_Vertical(row)(col),

                      Output           =>
Output(row)(col)

                      );

        end generate;

    end generate;

    Gen_Interior_Col : if col > 0 generate

        Gen_Top_Edge : if row = 0 generate
            PE_Inst : entity
work.Processing_Element(Behavioral)
                Generic map ( Input_Width      => Input_Width,
                      Accumulator_Width =>
Accumulator_Width )

                Port map      ( Clock           => Clock,
                      Reset             => Reset,
                      Enable            => Enable,
                      Clear_Accumulator =>
Clear_Accumulator,

                      Input_A          =>

```



```
        end generate;  
end generate;  
  
end Behavioral;
```